



DATA STRUCTURES

CONTENTS



15. Priority Queue and Heaps

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept of an important ADT – the Queue ADT. In this module we will discuss one very important application of Priority Queues – Heaps.

Learning Objectives

The learning objectives of the module are as follows:

- To explain the Concept of Heap and Priority Queue
- To discuss the properties of Binary Heap
- To describe the operations on a Binary Heap
- To outline the HeapSort algorithm

15.1 Introduction

Let us first recall the implementation of a priority queue using a simple linked list.

We can do insertion at the front which takes time $O(1)$ but in this case the deletion of the minimum takes $O(N)$. The other method is to keep list sorted where insertion takes time $O(N)$ while the important operation of a priority queue `deleteMin` takes time $O(1)$. In this module we will discuss an array implementation called Binary Heap which does not require links and will support both insertion and `deleteMin` operations in $O(\log N)$ worst-case time.

15.2 Implementation of Priority Queues

In this module we will discuss the implementation of priority queues using **heaps**. **Heap** is essentially a complete binary tree and hence can be efficiently represented using array-based representation. Priority queues can also be implemented using **leftist trees** which is basically a linked list implementation.

15.2.1 Binary Heap

A **binary heap** is the implementation of a heap using a **binary tree**. A binary heap is basically a **binary tree** with **two** additional **constraints**:

- **Shape (Structure) property:** A binary heap is a **complete binary tree** that is only the last level of the binary tree may be incomplete. A **heap** is a **complete binary tree**, represented as an array. A **complete binary tree** is a tree where all the nodes have two children that is completely filled, except probably at the lowest level where the nodes are filled in the left to right order. The **complete binary tree** of height h has between 2^h and $2^{h+1} - 1$ nodes. For a given number of nodes N , the height of a complete binary tree is $\lceil \log N \rceil$. This property allows the efficient implementation of the binary heap using an array such that:
- For any element in array position i , the left child is in position $2i$ and the right child is in the position after the left child ($2i + 1$). This in turn means that the parent of a node i is in position $\lfloor i/2 \rfloor$.
- **Heap-Order Property:** In a heap, for every node X with parent P , the key in P is smaller than or equal to the key in X . This ensures that the minimum element is always at the root. Thus the operation **findMin** can be carried out in constant time. This type of heap where the minimum element is always at the root is called the **min heap**. A **max heap** supports access of the maximum element instead of the minimum at the root of the tree. Therefore we describe the heap

as a data structure that satisfies the heap property. The heap property states that if the node A is the **parent** of node B, the key value of **A** is **ordered** with respect value of **B**. The above ordering applies to every node A in the tree.

15.3 Binary Heap

A **max tree (min tree)** is a tree in which the value in each node is **greater (less)** **than or equal** to those in its children if they exist. Note that there is no ordering among the children. In general the max or min tree need not be a binary tree but however the heap property has to be satisfied. On the other hand a **max binary heap (min binary heap)** is a **max (min) binary tree** that is also a **complete binary tree**. Figure 15.1 shows the example of a Max Heap.

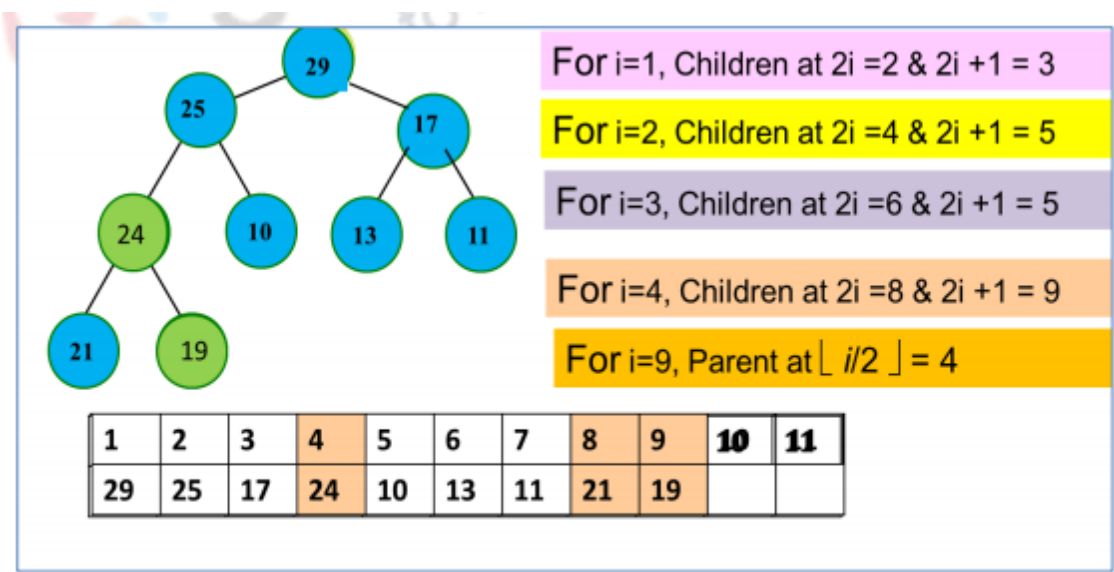


Figure 15.1 Binary Heap - Example : Max Heap

The Max Heap follows the two constraints namely the shape property and the heap property. Since we are discussing a Max heap, the highest priority item is the maximum key and this element will be the root of the tree. As can be seen from Figure 15.1 due to the shape property, the tree is complete except at the last or final level. This enables the heap to be stored using arrays. In our example, the root has the largest element 29 and this element is stored in the first location of the array. Now for $i=1$, the children are at $2i$ and $2i+1$, that is 29's children – 25 and 17 are

stored in array locations 2 and 3 respectively. For $i=2$, that is 25's children 24 and 10 are stored in array locations 4 ($2i-2*2$) and 5 ($2i+1-2*2+1$) respectively. Similarly for $i=3$ that is 17's children 13 and 11 are stored in array locations 6 and 7 while for $i=4$, 24's children are stored in locations 8 and 9 respectively.

An example of a Min heap where the minimum element is stored at the root and where the children are greater than parent is shown in Figure 15.2.

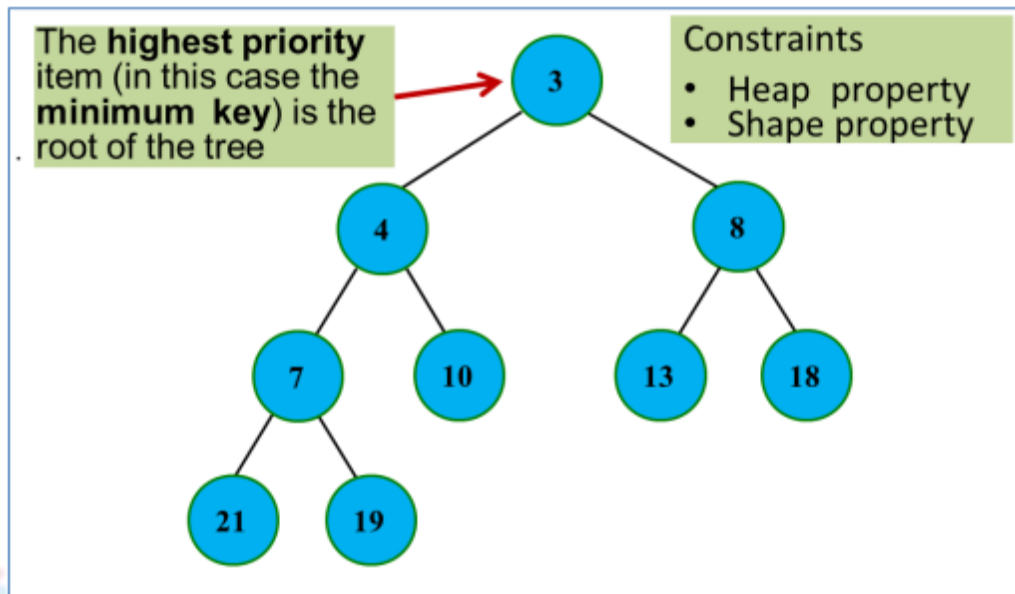


Figure 15.2 Binary Heap - Example: Min Heap

15.3.1 Binary Heap: *FindMin/Max()*

The example we will consider for discussing the different operations on the binary heap is a Min heap. As we already discussed the highest priority item is at the root of the binary tree and hence the **FindMin()** has to just return the root where running time is $O(1)$.

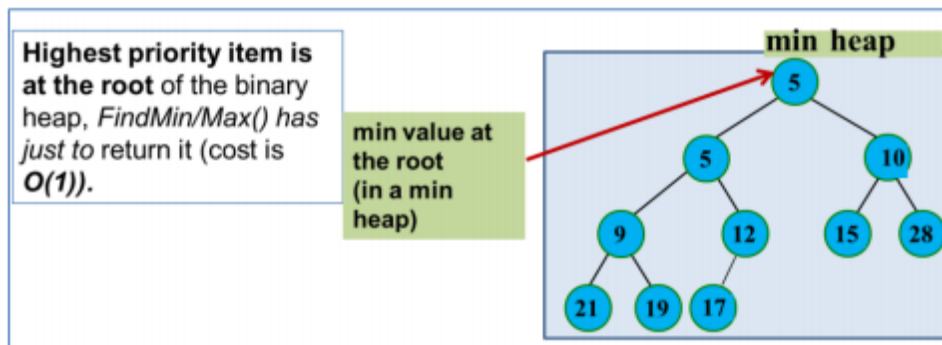
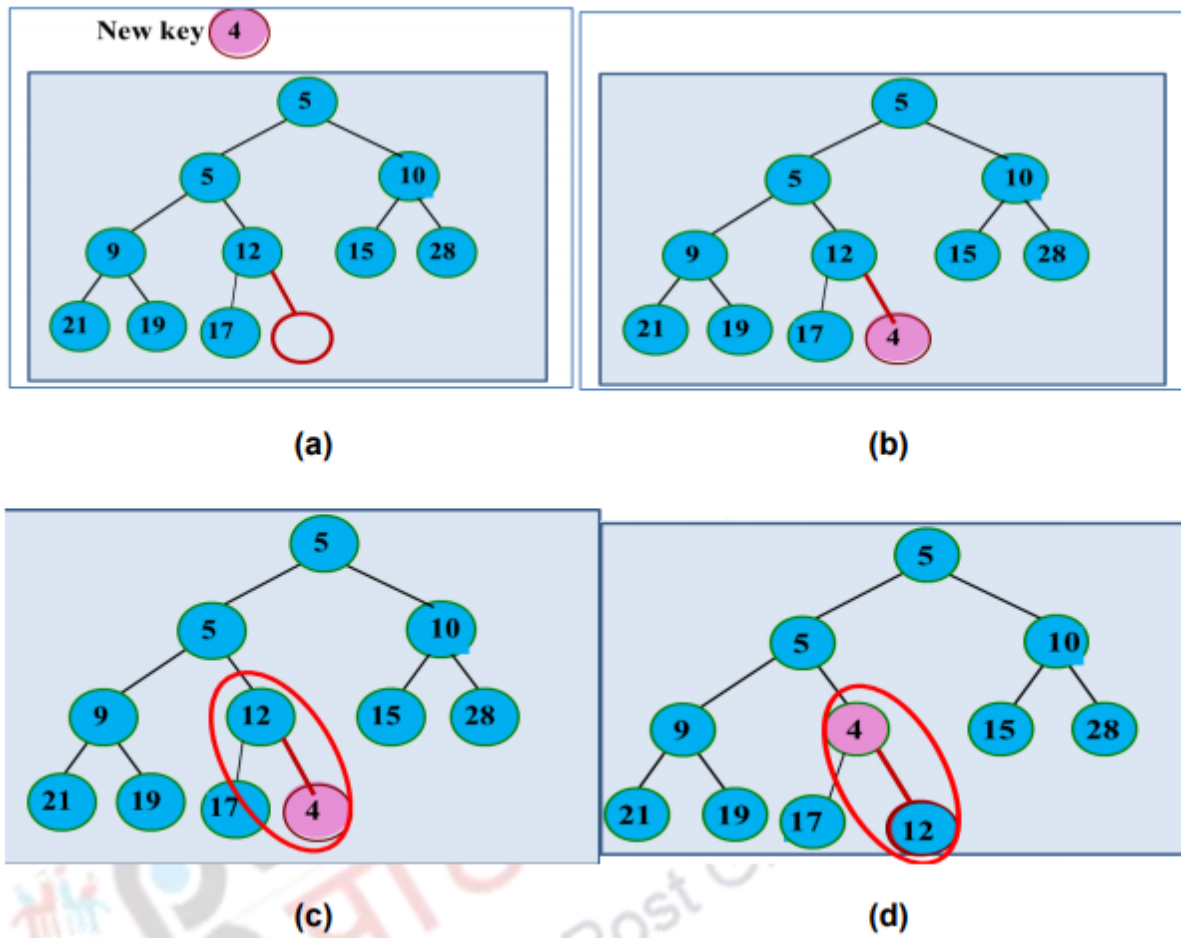


Figure 15.3 *FindMin*

15.3.2 Binary Heap: *Insert(key)*



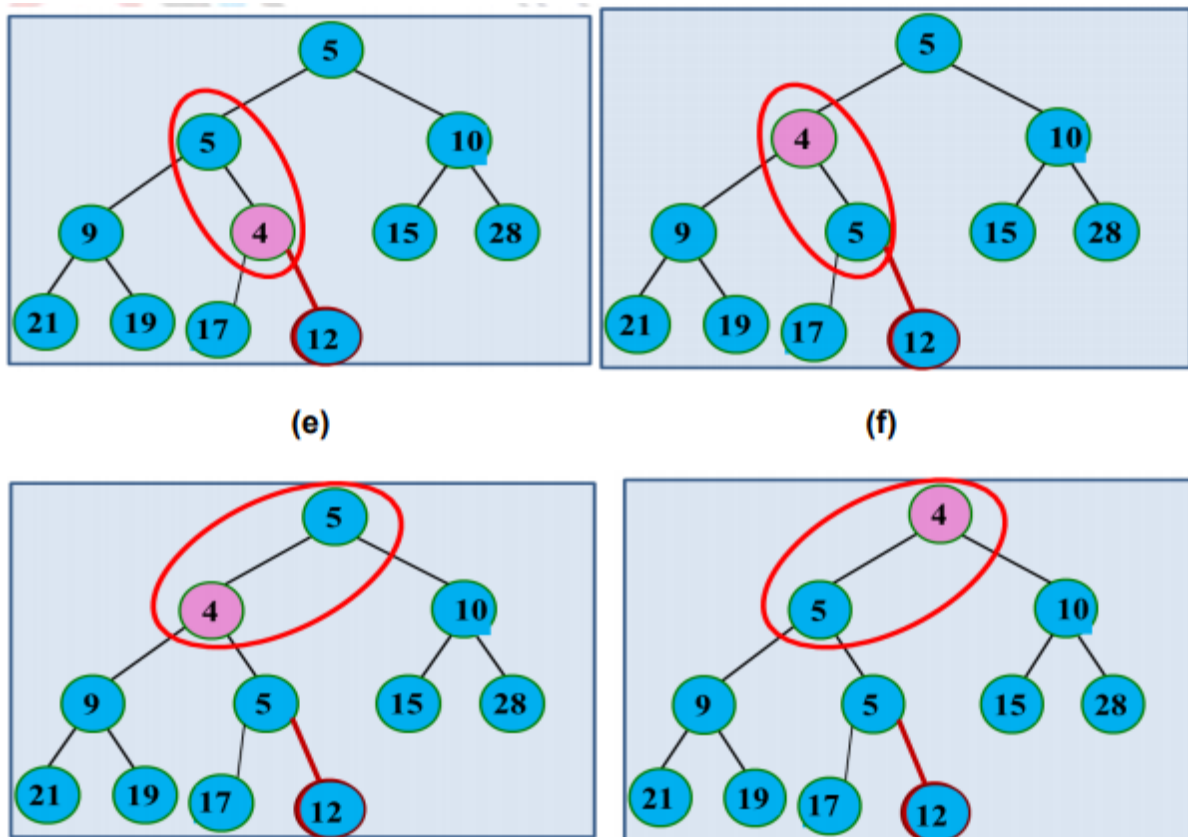


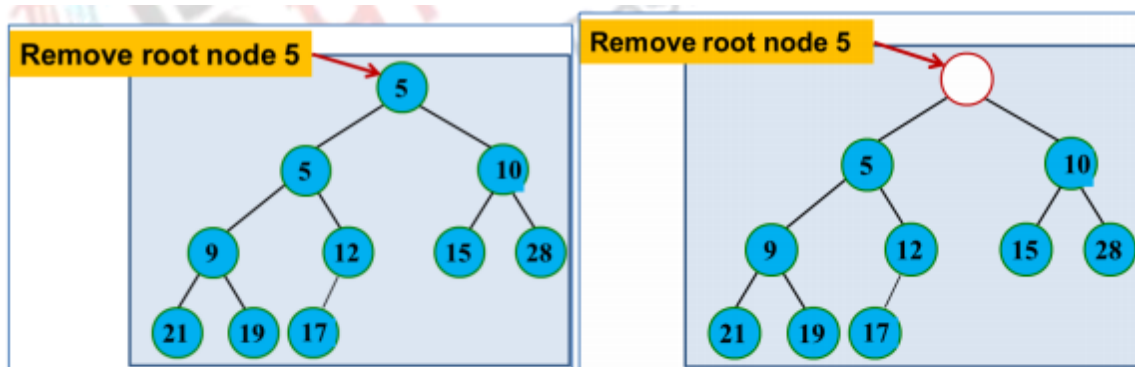
Figure 15.4 *Insert(key)*

A new item is always inserted in the last level of the tree, in the first free spot from left to right in the complete binary tree. The insertion of this new item most probably violates the heap property. Step 2 is when this new item ‘Bubbles up’ until the heap property is satisfied again. Figure 15.4 (c) and (d) shows the checking of the Min property and the exchange of the new element 4 with its parent 12 since this being a Min heap, parent element should be less than the child. Figure 15.4 (e) and (f) shows the checking of the Min property and the exchange of the element 4 with its new parent 5 since this being a Min heap, parent element should be less than the child. Figure 15.4 (g) and (h) shows the checking of the Min property and the exchange of the element 4 with its new parent (again) 5 since this being a Min heap, parent element should be less than the child. Now the new element has bubbled up the tree so that the Min heap property is satisfied. In our example 4 becomes the new root.

15.3.3 Binary Heap: Delete()

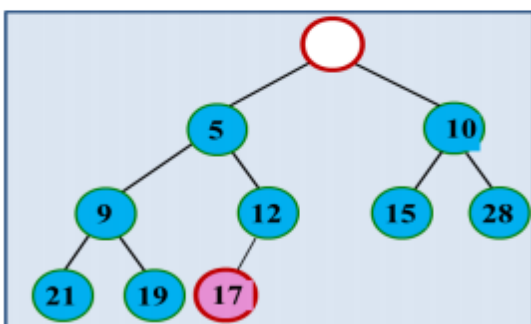
Figure 15.5 shows the deletion of an item, usually the minimum item that will at the root of the Min heap. Figure 15.5 (a) and (b) shows step 1 where here remove root item. Now in Step 2 we replace the root with the last item, from left to right, in the

last level (Figure 15.5 (c) and (d)). Now we go to Step 3 where the new root item 'Bubbles down' until the heap property is satisfied. When swapping a parent, we must always swap with the minimum child. Figure 15.5 (e) to (h) shows the bubbling down of node till the heap property is satisfied.

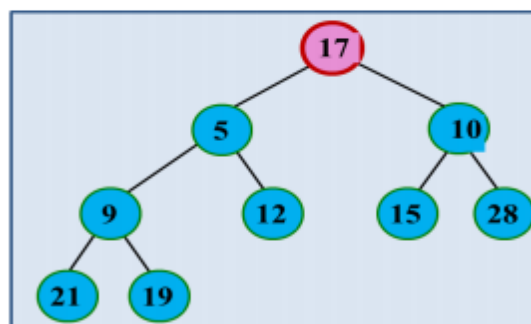


(a)

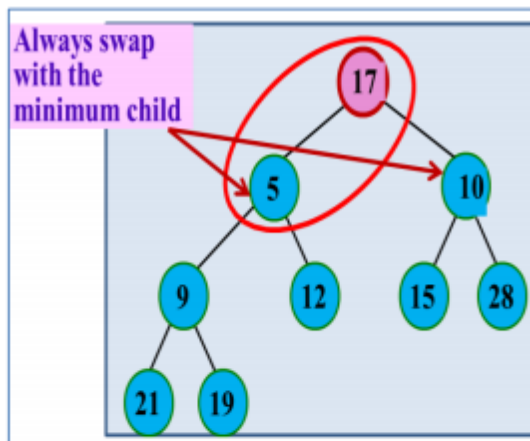
(b)



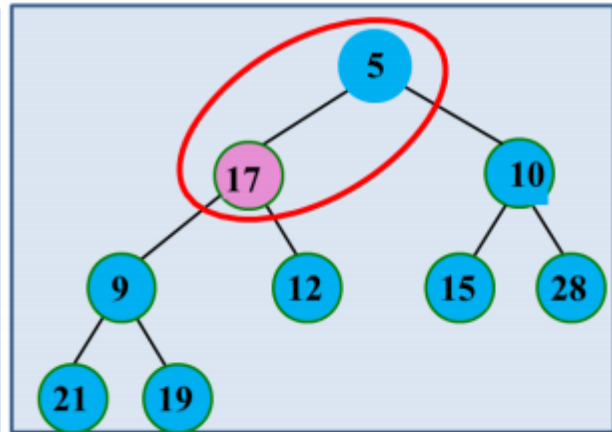
(c)



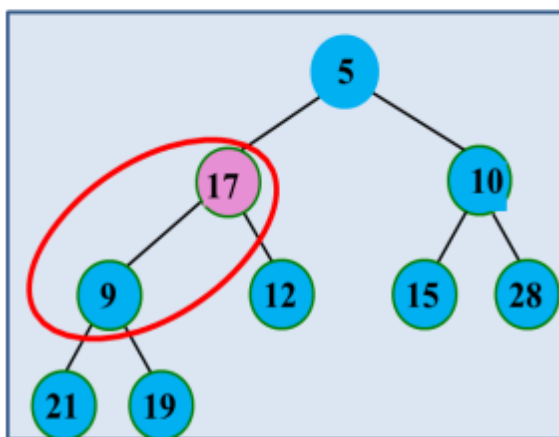
(d)



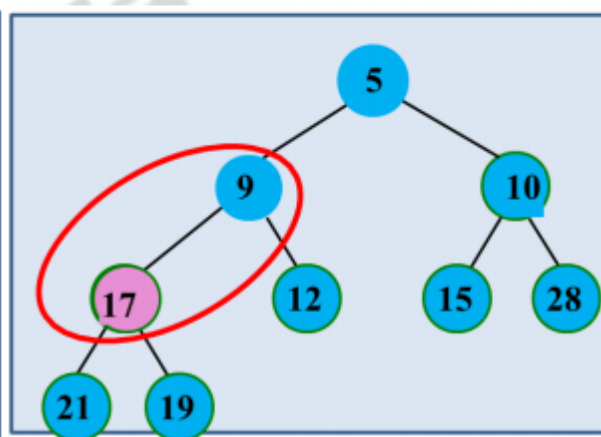
(e)



(f)



(g)



(h)

15.4 Cost of Binary Heap Construction

The **insertion** of each new item into a binary heap costs $O(\log n)$. If there are n items to be inserted, the overall cost to build the whole binary heap will be $O(n \log n)$. For **analysing buildHeap**, the linear time bound can be shown by computing the sum of the heights of all the nodes in the heap. The perfect binary tree of height h contains $N = 2^{h+1} - 1$ nodes.

15.5 Heapsort

The priority queue can be used to sort N items by inserting every item into a

binary heap and extracting every item by calling **RemoveMin** N times, thus sorting the result. An algorithm based on this idea of using heap is the *heapsort* algorithm. This algorithm is an $O(N \log N)$ worst-case sorting algorithm. The algorithm uses an extra array for the items exiting the heap. We can avoid this problem by shrinking the heap by 1 after each RemoveMin. In this way the cell that was last in the heap can be used to store the element that was just deleted. Using this strategy, after the last RemoveMin, the array will contain all elements in *decreasing* order if the heap used was a Min heap. In case we want the elements to be sorted in *increasing* order we must use a *max heap*.

Summary

- Explained the Concept of Heap and Priority Queue
- Explained the concept of MinHeap & MaxHeap
- Discussed the structure and order properties Binary Heap
- Described the operations on a Binary Heap
- Outlined the HeapSort algorithm

you can view video on Priority Queue and Heaps



Web Links
https://www.cs.princeton.edu/~rs/AlgsDS07/06PriorityQueues.pdf
https://people.mpi-inf.mpg.de/~mehlhorn/ftp/.../PriorityQueues.pdf
www.cs.cmu.edu/~adamchik/15-121/lectures/.../heaps.html
https://www.cs.purdue.edu/homes/ayg/CS251/slides/chap6.pdf
https://www.cs.cmu.edu/~wlovas/15122-r11/lectures/15-priorqs.pdf
www.eecs.wsu.edu/~ananth/CptS223/Lectures/heaps.pdf
www.inf.ed.ac.uk/teaching/courses/inf2b/algnotes/note06.pdf
http://www.slideshare.net/jsaddam709/deque-and-its-applications
dpnm.postech.ac.kr/cs233/lecture/lecture9.ppt
Supporting & Reference Materials
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Adam Drozdek, "Data Structures and Algorithms in C++", Cengage; 4th edition, 2013
Alfred V. Aho and Jeffrey D. Ullman, "Data Structures and Algorithms", 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014